Engenharia de Software – Aula 24

Sprint 04: Casos de Uso, Diagrama de Classes e Início da Codificação

Prof. Me. Reginaldo Pereira Fernandes

Instituto Federal de Educação, Ciência e Tecnologia do Ceará – Campus Tauá
Técnico em Informática para Internet (TSII.210)

Semana 1 – Encontro 1 (100 min)

Agenda do Encontro 1 (100 Minutos)

1. Contexto da Sprint e Liderança Técnica
2. Diagrama de Casos de Uso (UML 2)
3. Diagrama de Classes (UML 2)
4. Arquitetura do Projeto BiblioTauá
5. Implementação: Exceções de Domínio
6. Implementação: Modelo Cupom
7. Rito Ágil e Próximos Passos

Cenário Corporativo: O Sistema BiblioTauá

- **O Sistema:** Livraria Universitária e Biblioteca do IFCE Campus Tauá.
- **Status Atual (v0.3.0):**
  - Módulo de Catálogo de Livros e Estoque funcional.
  - Cadastro de Clientes e Discentes ativo.
  - Persistência relacional em banco de dados SQLite.
- **O Desafio da Sprint 04:**
  - Feira do Livro do IFCE se aproximando.
  - Demanda de negócio: conceder **Cupons Promocionais** aos estudantes.
  - Meta: Entregar o Módulo de Cupons em 2 semanas (4 encontros).

Sprint Planning: Fala da Tech Lead Aline Mendes

Diretriz da Liderança Técnica

*"Mariana assume a interface web. Lucas adapta o backend de pagamentos para o PIX. E **VOCÊ** assume a tarefa principal: **TASK-204 - Motor de Validação e Aplicação de Cupons**."*

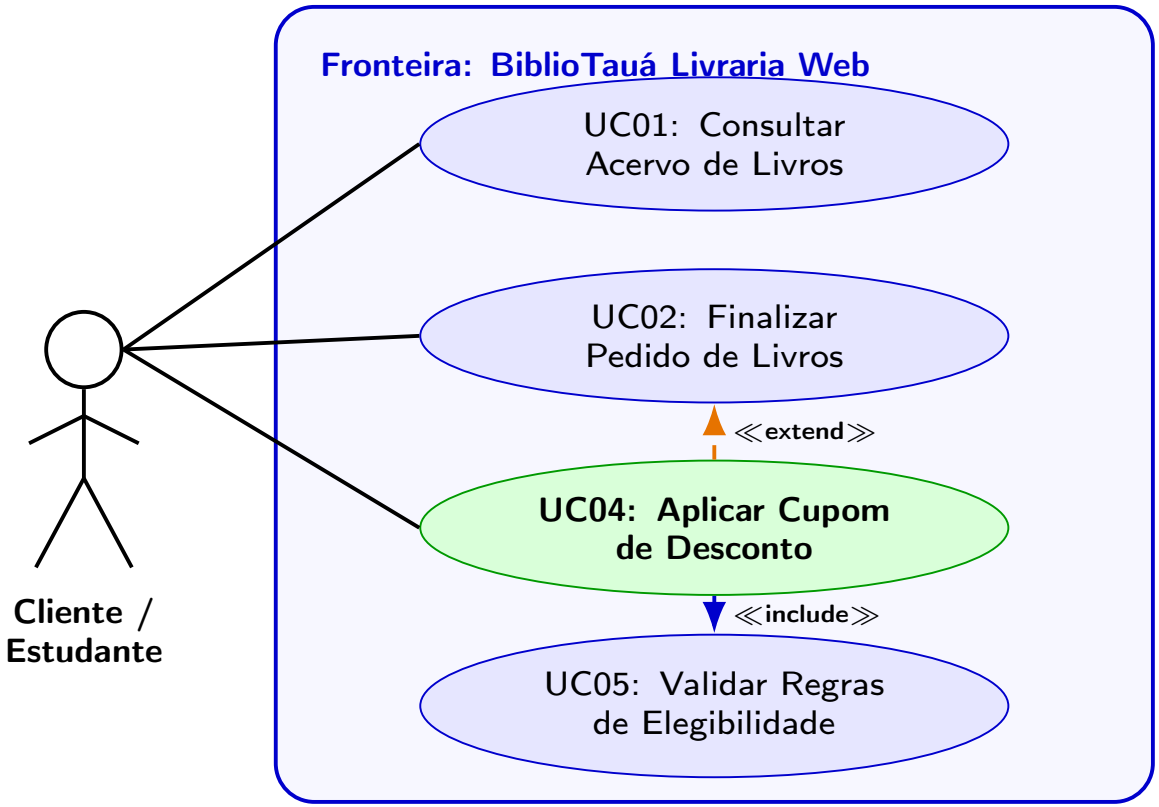
Ritmo Ágil de Alta Produtividade

Neste primeiro encontro, finalizaremos os diagramas de Casos de Uso e Classes e **já iniciaremos a codificação em Python**!

O Ticket da Sprint: TASK-204

Campo	Descrição do Ticket
Identificador	**TASK-204**
Título	Motor de Validação e Aplicação de Cupons no Checkout
Responsável	Desenvolvedor Júnior (Discente)
Prioridade	Alta / Bloqueante para a Sprint 04
Estimativa	4 Encontros de Trabalho (8 Horas-Aula)
Branch Git	`feature/task-204-motor-cupons`
Entrega Hoje	DUC $\rightarrow$ Diagrama de Classes $\rightarrow$ Exceções & Modelo Cupom

Modelagem Comportamental: Diagrama de Casos de Uso



Guia de Leitura UML:

≪extend≫ (Extensão):
Condicional: o cliente só aplica cupom se desejar durante o checkout.

≪include≫ (Inclusão):
Obrigatório: ao aplicar cupom, o sistema valida compulsoriamente as regras.

Foco da TASK-204:

Motor de regras do **UC04**.

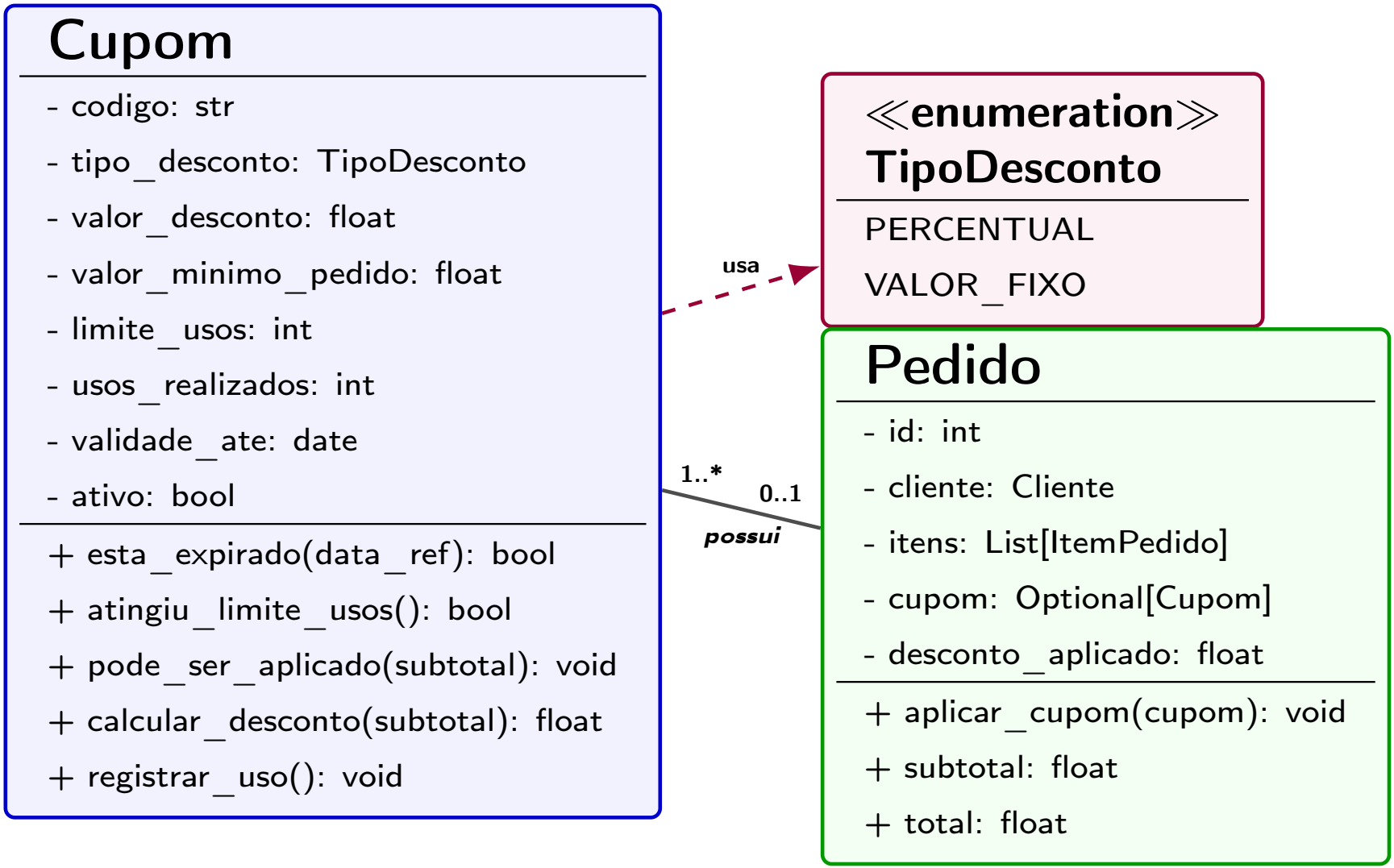
Dissecando a DUC UC04: Regras de Negócio e Exceções

Fluxo	Cenário / Regra de Negócio	Exceção Disparada em Python
Principal	**Cupom ativo, dentro do prazo, cota disponível**	**Desconto calculado e total abatido**
FA01	**Desconto percentual (% sobre o subtotal)**	`desconto = subtotal * (valor / 100)`
FA02	**Desconto em valor fixo (R$)**	`desconto = min(valor, subtotal)`
FE01	**Cupom inexistente ou inativo (`ativo=0`)**	`CupomInexistenteError` / `CupomInvalidoError`
FE02	**Data atual ultrapassa o limite de validade**	`CupomExpiradoError`
FE03	**Usos realizados atingiram o limite**	`CupomEsgotadoError`
FE04	**Subtotal da compra menor que o mínimo exigido**	`ValorMinimoNaoAtingidoError`

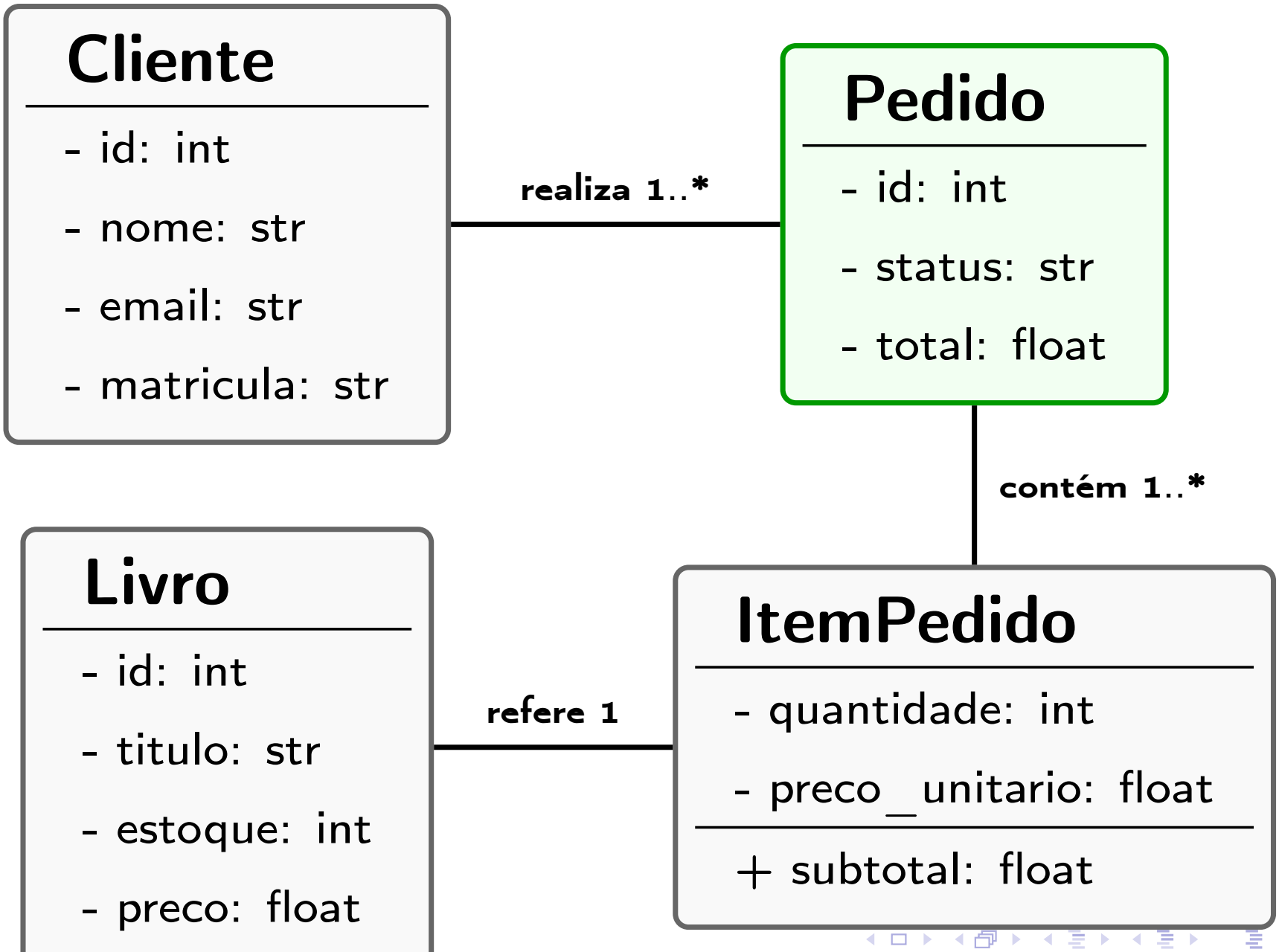
Conexão Direta: Especificação → Código

Cada fluxo de exceção (FE01 a FE04) mapeia exatamente para uma classe em `exceptions.py`!

Modelagem Estrutural: Diagrama de Classes do Domínio



Visão Geral das Classes da Livraria BiblioTauá



Arquitetura do Projeto: Estrutura de Pastas e Módulos

Árvore de Arquivos do BiblioTauá

```
bibliotaua/
|-- database.py            (SQLite v0.3.0)
|-- exceptions.py          (NOVO - Aula 24)
|-- models/
|   |-- cliente.py         (Base v0.3.0)
|   |-- livro.py           (Base v0.3.0)
|   |-- item_pedido.py     (Base v0.3.0)
|   |-- pedido.py          (Base v0.3.0)
|   `-- cupom.py           (NOVO - Aula 24)
`-- repositories/
    |-- livro_repo.py      (Base v0.3.0)
    `-- pedido_repo.py     (Base v0.3.0)
```

Papel de Cada Camada

- `models/`: Regras de negócio e entidades OO puras.
- `repositories/`: Consultas SQL e persistência SQLite.
- `database.py`: Gerenciamento de conexão e DDL.
- `exceptions.py`: Exceções mapeadas da DUC.

Por que o Repositório tem Mais Códigos que os Slides?

Simulação Real de Engenharia de Software Ágil

Em um time profissional, um novo ticket de Sprint nunca começa com o repositório em branco!

1. Base Pré-existente (v0.3.0)

- Construída nas Sprints 01 a 03.
- Contém as entidades que já funcionam: `Cliente`, `Livro`, `ItemPedido`, `Pedido`.
- Evita erros de importação (`ModuleNotFoundError`) e viabiliza a execução do sistema completo.

2. O Nosso Foco Hoje (TASK-204)

- Baixe o `projeto_aluno.zip` com os arquivos base e marcações `TODO`.
- `exceptions.py`: Criar classes de erro da DUC.
- `models/cupom.py`: Criar métodos de regra.
- Teste criando o script `main.py` no projeto!

Mão na Massa: Hierarquia de Exceções (exceptions.py)

```python
class BiblioTauaException(Exception):
    """Excecao base para regras de negocio do BiblioTaua."""
    pass

class CupomInvalidoError(BiblioTauaException):
    pass

class CupomInexistenteError(CupomInvalidoError):
    def __init__(self, codigo: str):
        super().__init__(f"Cupom '{codigo}' nao encontrado no sistema.")

class CupomInativoError(CupomInvalidoError):
    def __init__(self, codigo: str):
        super().__init__(f"O cupom '{codigo}' encontra-se desativado temporariamente.")

class CupomExpiradoError(CupomInvalidoError):
    def __init__(self, codigo: str, validade_ate: str):
        super().__init__(f"O cupom '{codigo}' expirou em {validade_ate}.")

class CupomEsgotadoError(CupomInvalidoError):
    def __init__(self, codigo: str, limite: int):
        super().__init__(f"O cupom '{codigo}' atingiu a quantidade maxima de {limite} usos
            .")

class ValorMinimoNaoAtingidoError(CupomInvalidoError):
    def __init__(self, codigo: str, valor_minimo: float, subtotal_atual: float):
        super().__init__(
            f"O cupom '{codigo}' exige pedido minimo de R$ {valor_minimo:.2f}. "
            f"Subtotal atual: R$ {subtotal_atual:.2f}."
        )
```

Mão na Massa: Entidade Cupom (models/cupom.py)

```python
from dataclasses import dataclass
from datetime import date
from enum import Enum
from typing import Optional
from bibliotaua.exceptions import *

class TipoDesconto(str, Enum):
    PERCENTUAL = "PERCENTUAL"
    VALOR_FIXO = "VALOR_FIXO"

@dataclass
class Cupom:
    codigo: str
    tipo_desconto: TipoDesconto
    valor_desconto: float
    valor_minimo_pedido: float = 0.0
    limite_usos: int = 100
    usos_realizados: int = 0
    validade_ate: date = date.today()
    ativo: bool = True
    id: Optional[int] = None
```

Mão na Massa: Métodos de Domínio na Entidade Cupom

```
    def pode_ser_aplicado_ao_pedido(self, subtotal: float, data_ref: Optional
        [date] = None) -> None:
        if not self.ativo:
            raise CupomInativoError(self.codigo)
        if self.esta_expirado(data_ref):
            validade_str = self.obter_data_validade().strftime("%d/%m/%Y")
            raise CupomExpiradoError(self.codigo, validade_str)
        if self.atingiu_limite_usos():
            raise CupomEsgotadoError(self.codigo, self.limite_usos)
        if subtotal < self.valor_minimo_pedido:
            raise ValorMinimoNaoAtingidoError(self.codigo, self.
                valor_minimo_pedido, subtotal)

    def calcular_desconto(self, subtotal: float, data_ref: Optional[date] =
        None) -> float:
        self.pode_ser_aplicado_ao_pedido(subtotal, data_ref)
        if self.tipo_desconto == TipoDesconto.PERCENTUAL:
            desconto = (subtotal * self.valor_desconto) / 100.0
        else:
            desconto = self.valor_desconto
        return round(min(desconto, subtotal), 2)
```

Testando a Implementação: Arquivo main.py

Criando o script de teste (main.py)

```
from datetime import date, timedelta
from bibliotaua.models.cupom import Cupom,
     TipoDesconto
from bibliotaua.exceptions import
     ValorMinimoNaoAtingidoError

def main():
    cupom = Cupom(
        codigo="BEMVINDO10",
        tipo_desconto=TipoDesconto.PERCENTUAL,
        valor_desconto=10.0,
        valor_minimo_pedido=50.0,
        validade_ate=date.today() + timedelta(
            days=5)
    )
    # 1. Pedido valido (R$ 100.00)
    d = cupom.calcular_desconto(100.0)
    print(f"Desconto: R$ {d:.2f}")

    # 2. Pedido abaixo do minimo (R$ 30.00)
    try:
        cupom.calcular_desconto(30.0)
    except ValorMinimoNaoAtingidoError as erro:
        print(f"Bloqueado: {erro}")

if __name__ == "__main__":
    main()
```

Execução no Terminal

```
$ python3 main.py

Desconto: R$ 10.00
Bloqueado: O cupom 'BEMVINDO10'
exige pedido minimo de R$ 50.00.
Subtotal atual: R$ 30.00.
```

Conclusão da Aula 24

A entidade Cupom e as regras de negócio foram validadas com sucesso pelo script main.py!

Rito Ágil: Registro da Daily 1

Anotação para o Relatório da Sprint no `guia_desenvolvedor.md`

- **O que realizei hoje?**
  - Interpretei o Diagrama de Casos de Uso e o Diagrama de Classes.
  - Mapeei todos os fluxos da DUC UC04 em regras técnicas.
  - Implementei a hierarquia de exceções em `exceptions.py`.
  - Implementei a entidade de domínio `Cupom` e enum `TipoDesconto` com validações no `models/cupom.py`.
- **O que farei no Encontro 2 (Aula 25)?**
  - Mapeamento relacional (DER): criação da tabela `cupons` no SQLite.
  - Implementação do padrão repositório em `CupomRepository`.
- **Impedimentos?** Nenhum impedimento.

Síntese do Encontro 1

- **Objetivo Cumprido com Sucesso:** Saímos do alinhamento ágil, modelamos o sistema em UML (UC + Classes) e entregamos os primeiros arquivos de código em produção.
- **Para o Encontro 2 (Aula 25):**
  - Persistência relacional com SQLite.
  - Script DDL e constraints `CHECK`, `UNIQUE`.
  - Repositório com prevenção contra SQL Injection.

Excelente início! O motor de cupons já começou a ganhar vida!